By Asif Raja

# PIZZA

By Asif Raja

# ABOUT SQL PROJECT

The objective of this project is to demonstrate strong SQL querying, data modeling, and analytical problem-solving skills by transforming raw sales data into actionable insights that can support business decisions such as menu optimization, peak-hour staffing, and revenue forecasting.

This project focuses on analyzing pizza sales data using SQL to uncover meaningful business insights related to customer ordering behavior, revenue generation, and product performance.

The dataset consists of four relational tables: orders, order_details, pizzas, and pizza_types, connected using primary and foreign key relationships to simulate a real-world transactional data.

@pizzapalace

By Asif Raja

# RETRIEVE THE TOTAL NUMBER OF ORDERS PLACED.

**CODE :**

```
SELECT
 COUNT(ORDER_ID) TOTAL_ORDERS
FROM
 ORDERS;
```

**OUTPUT :**

| | total_orders bigint |
|---|---|
| 1 | 21350 |

By Asif Raja

# CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES.

## CODE :

```
SELECT
 SUM(ORDER_DETAILS.QUANTITY * PIZZAS.PRICE) AS
TOTAL_REVENUE
FROM
 ORDER_DETAILS
 JOIN PIZZAS ON PIZZAS.PIZZA_ID =
ORDER_DETAILS.PIZZA_ID;
```

## OUTPUT :

| | total_revenue numeric |
|---|---|
| 1 | 817860.05 |

By Asif Raja

# IDENTIFY THE HIGHEST-PRICED PIZZA.

## CODE :

```
SELECT
 *
FROM
 PIZZAS
ORDER BY
 PRICE DESC
LIMIT
 1;
```

## OUTPUT :

| pizza_id [PK] character varying (100) | pizza_type_id character varying (100) | size character varying (50) | price numeric (10,2) |
|---|---|---|---|
| the_greek_xxl | the_greek | XXL | 35.95 |

By Asif Raja

# IDENTIFY THE MOST COMMON PIZZA SIZE ORDERED.

## CODE :

```
SELECT
 P.SIZE,
 COUNT(P.SIZE) AS COMMON_SIZE
FROM
 PIZZAS P
 JOIN ORDER_DETAILS O ON P.PIZZA_ID = O.PIZZA_ID
GROUP BY
 P.SIZE
ORDER BY
 COMMON_SIZE DESC
LIMIT  1;
```

## OUTPUT :

| size<br>character varying (50) | common_size<br>bigint |
|---|---|
| L | 18526 |

By Asif Raja

# LIST THE TOP 5 MOST ORDERED PIZZA TYPES ALONG WITH THEIR QUANTITIES.

## CODE :

```
SELECT
 PIZZAS.PIZZA_TYPE_ID,
 COUNT(ORDER_DETAILS.QUANTITY) AS
PIZZA_QUANTITY
FROM PIZZAS
 JOIN ORDER_DETAILS ON PIZZAS.PIZZA_ID =
ORDER_DETAILS.PIZZA_ID
GROUP BY
 PIZZAS.PIZZA_TYPE_ID
ORDER BY PIZZA_QUANTITY DESC
LIMIT 5;
```

## OUTPUT :

| pizza_type_id character varying (100) | pizza_quantity bigint |
|---|---|
| classic_dlx | 2416 |
| bbq_ckn | 2372 |
| hawaiian | 2370 |
| pepperoni | 2369 |
| thai_ckn | 2315 |

By Asif Raja

# JOIN THE NECESSARY TABLES TO FIND THE TOTAL QUANTITY OF EACH PIZZA CATEGORY ORDERED.

## CODE :

```
SELECT
 PIZZA_TYPES.CATEGORY,
 SUM(ORDER_DETAILS.QUANTITY) AS PIZZA_QUANTITY
FROM PIZZA_TYPES
 JOIN PIZZAS ON PIZZA_TYPES.PIZZA_TYPE_ID =
PIZZAS.PIZZA_TYPE_ID
 JOIN ORDER_DETAILS ON PIZZAS.PIZZA_ID =
ORDER_DETAILS.PIZZA_ID
GROUP BY PIZZA_TYPES.CATEGORY
ORDER BY
 PIZZA_QUANTITY DESC;
```

## OUTPUT :

| category character varying (100) | pizza_quantity bigint |
|---|---|
| Classic | 14888 |
| Supreme | 11987 |
| Veggie | 11649 |
| Chicken | 11050 |

By Asif Raja

# DETERMINE THE DISTRIBUTION OF ORDERS BY HOUR OF THE DAY.

## CODE :

```
SELECT
EXTRACT(HOUR FROM ORDER_TIME) AS ORDER_HOUR,
COUNT(ORDER_ID) AS ORDERS_RECIEVED
FROM ORDERS
GROUP BY
EXTRACT(HOUR FROM ORDER_TIME)
ORDER BY
 ORDER_HOUR ASC;
```

## OUTPUT :

| | order_hour numeric | orders_recieved bigint |
|---|---|---|
| 1 | 9 | 1 |
| 2 | 10 | 8 |
| 3 | 11 | 1231 |
| 4 | 12 | 2520 |
| 5 | 13 | 2455 |
| 6 | 14 | 1472 |
| 7 | 15 | 1468 |
| 8 | 16 | 1920 |
| 9 | 17 | 2336 |
| 10 | 18 | 2399 |
| 11 | 19 | 2009 |
| 12 | 20 | 1642 |
| 13 | 21 | 1198 |
| 14 | 22 | 663 |
| 15 | 23 | 28 |

By Asif Raja

# JOIN RELEVANT TABLES TO FIND THE CATEGORY-WISE DISTRIBUTION OF PIZZAS.

## CODE :

```
SELECT
    pt.category,
    COUNT(p.pizza_id) AS total_pizzas
FROM pizzas p
JOIN pizza_types pt
    ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category
ORDER BY total_pizzas DESC;
```

## OUTPUT :

| category<br>character varying (100) | total_pizzas<br>bigint |
|---|---|
| Veggie | 27 |
| Classic | 26 |
| Supreme | 25 |
| Chicken | 18 |

By Asif Raja

# GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE NUMBER OF PIZZAS ORDERED PER DAY.

## CODE :

```
SELECT
 ROUND(AVG(QUANTITY), 0)
FROM
(SELECT
  ORDERS.ORDER_DATE,
  SUM(ORDER_DETAILS.QUANTITY) AS QUANTITY
 FROM
  ORDER_DETAILS
  JOIN ORDERS ON ORDER_DETAILS.ORDER_ID =
ORDERS.ORDER_ID
 GROUP BY
  ORDERS.ORDER_DATE) AS ORDER_QUANTITY;
```

## OUTPUT :

| | round numeric |
|---|---|
| 1 | 138 |

By Asif Raja

# DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE.

## CODE :

```
SELECT
 PIZZAS.PIZZA_TYPE_ID,
 SUM(PIZZAS.PRICE) AS PIZZA_REVENUE
FROM PIZZAS
 JOIN ORDER_DETAILS
 ON PIZZAS.PIZZA_ID = ORDER_DETAILS.PIZZA_ID
GROUP BY
PIZZAS.PIZZA_TYPE_ID
ORDER BY
 PIZZA_REVENUE DESC
LIMIT 3;
```

## OUTPUT :

| pizza_type_id character varying (100) | pizza_revenue numeric |
|---|---|
| thai_ckn | 42332.25 |
| bbq_ckn | 41683.00 |
| cali_ckn | 40166.50 |

By Asif Raja

# CALCULATE THE PERCENTAGE CONTRIBUTION OF EACH PIZZA TYPE TO TOTAL REVENUE.

## CODE :

```
SELECT PT.NAME,
 ROUND(
  SUM(OD.QUANTITY * P.PRICE) * 100 /
SUM(SUM(OD.QUANTITY * P.PRICE)) OVER (), 2)
 AS REVENUE_PERCENT
FROM ORDER_DETAILS OD
 JOIN PIZZAS P ON OD.PIZZA_ID = P.PIZZA_ID
 JOIN PIZZA_TYPES PT ON PT.PIZZA_TYPE_ID =
P.PIZZA_TYPE_ID
GROUP BY PT.NAME
ORDER BY
 REVENUE_PERCENT DESC;
```

## OUTPUT :

| name<br>character varying (100) | revenue_percent<br>numeric |
|---|---|
| The Thai Chicken Pizza | 5.31 |
| The Barbecue Chicken Pizza | 5.23 |
| The California Chicken Pizza | 5.06 |
| The Classic Deluxe Pizza | 4.67 |
| The Spicy Italian Pizza | 4.26 |
| The Southwest Chicken Pizza | 4.24 |
| The Italian Supreme Pizza | 4.09 |
| The Four Cheese Pizza | 3.95 |
| The Hawaiian Pizza | 3.95 |
| The Sicilian Pizza | 3.78 |

By Asif Raja

# ANALYZE THE CUMULATIVE REVENUE GENERATED OVER TIME.

## CODE :

```
SELECT  order_date,
   daily_revenue,
   SUM(daily_revenue) OVER (ORDER BY order_date)
AS cumulative_revenue
FROM (
   SELECT  o.order_date,
      SUM(od.quantity * p.price) AS daily_revenue
   FROM orders o
   JOIN order_details od
      ON o.order_id = od.order_id
   JOIN pizzas p
      ON od.pizza_id = p.pizza_id
   GROUP BY o.order_date
) daily_sales
ORDER BY order_date;
```

## OUTPUT :

| order_date date | daily_revenue numeric | cumulative_revenue numeric |
|---|---|---|
| 2015-01-01 | 2713.85 | 2713.85 |
| 2015-01-02 | 2731.90 | 5445.75 |
| 2015-01-03 | 2662.40 | 8108.15 |
| 2015-01-04 | 1755.45 | 9863.60 |
| 2015-01-05 | 2065.95 | 11929.55 |
| 2015-01-06 | 2428.95 | 14358.50 |
| 2015-01-07 | 2202.20 | 16560.70 |
| 2015-01-08 | 2838.35 | 19399.05 |
| 2015-01-09 | 2127.35 | 21526.40 |
| 2015-01-10 | 2463.95 | 23990.35 |

By Asif Raja

# DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE FOR EACH PIZZA CATEGORY.

## CODE :

```
SELECT NAME, REVENUE FROM
 (SELECT CATEGORY, NAME, REVENUE,
   RANK() OVER(PARTITION BY CATEGORY ORDER BY
REVENUE DESC) AS PR
  FROM
  (SELECT PT.CATEGORY, PT.NAME,
    SUM((OD.QUANTITY)*P.PRICE) AS REVENUE
   FROM
    PIZZAS P
   JOIN PIZZA_TYPES PT
    ON PT.PIZZA_TYPE_ID=P.PIZZA_TYPE_ID
   JOIN ORDER_DETAILS OD
    ON P.PIZZA_ID = OD.PIZZA_ID
GROUP BY
 PT.CATEGORY, PT.NAME) AS A) AS B
WHERE PR>=3;
```

## OUTPUT :

| name character varying (100) | revenue numeric |
|---|---|
| The California Chicken Piz... | 41409.50 |
| The Southwest Chicken Pi... | 34705.75 |
| The Chicken Alfredo Pizza | 16900.25 |

By Asif Raja

# THANK YOU

This project is created using PostGreSQL to provide better understanding  and uses of advance SQL concepts such as JOINs, CTEs, aggregate functions, subqueries, window functions, ranking functions, and date-time analysis.

@pizzapalace